

Zakura TLS RPC Handshake Connection Exhaustion Denial-of-Service Vulnerability Report

Summary

Zakura's TLS RPC accept loop spawns one task per TCP connection and waits for a TLS handshake without a timeout or global limit. Anonymous clients can stay silent, retain sockets and tasks, exhaust file descriptors, and make legitimate RPC traffic fail before authentication is reached.

Mitigating Factors:

- Only deployments with `rpc.tls` enabled use this handshake path.
- A loopback listener, strict firewall, or front proxy with connection controls substantially reduces reachability.
- The confirmed consequence is service unavailability rather than confidentiality or transaction-integrity loss.
- An external TLS terminator with handshake deadlines can provide an immediate operational boundary.

Aggravating Factors:

- The attack happens before cookie authentication, so no RPC credentials are needed.
- `acceptor.accept(socket).await` has no deadline; a silent peer can remain pending indefinitely.
- The server spawns before acquiring any per-client or global connection permit.
- Shutdown handling is reached only after a successful handshake, so stalled tasks resist graceful shutdown.
- An `EMFILE` accept error can terminate the RPC task and propagate into node shutdown.

Vulnerability Details:

Status, Product, and Version

- Record status: PUBLISHED
- Product and version: Zakura / zakurad / 1.2.0
- Weakness classification: CWE-400: Uncontrolled Resource Consumption; CWE-770: Allocation of Resources Without Limits or Throttling

Cross-Validation Conclusion

The pre-authentication TLS handshake path has no time or concurrency boundary. A reachable attacker can accumulate pending handshakes until legitimate connections fail and the RPC service or node exits because file descriptors are exhausted.

Trigger Conditions

- `rpc.tls` is enabled and the listener is reachable.
- The attacker can establish TCP connections but need not complete TLS.
- Connections are created faster than the operating system or application releases resources.
- No front proxy or firewall enforces a lower connection and handshake budget.

Normal and Abnormal Behavior

Normal behavior

A client completes TLS within a short deadline, reaches RPC authentication, and releases its connection resources when the request or session ends.

Abnormal behavior

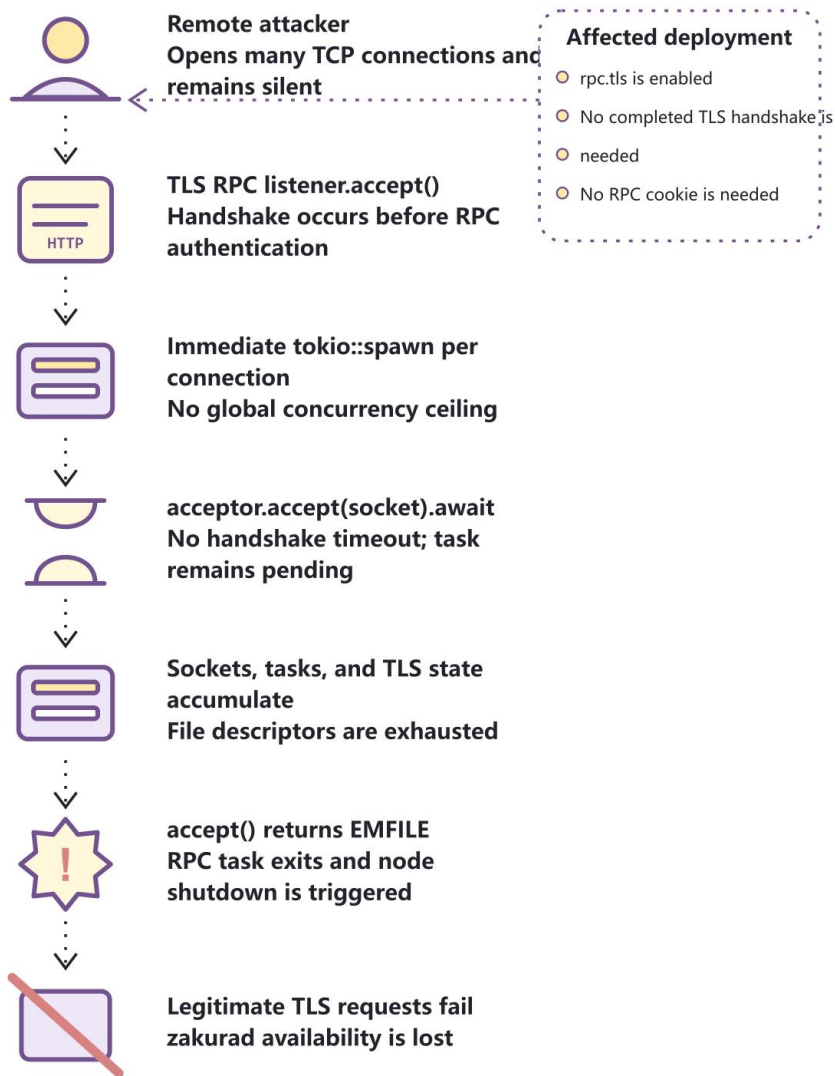
A silent client never reaches authentication, yet its socket, task, and TLS state remain allocated until process or network cleanup.

Full Call Chain Diagram

The complete call chain is shown below. The dashed box states the deployment condition, version boundary, or evidence boundary that controls the conclusion.

TLS RPC Handshake-Exhaustion Call Chain

Silent connections retain tasks and file descriptors before authentication



Technical Details:

Root cause 1: one unbounded task is created per accepted socket

```
while let Ok((socket, peer)) = listener.accept().await {
    let acceptor = acceptor.clone();
    tokio::spawn(async move {
        let tls_stream = acceptor.accept(socket).await?;
        serve_rpc(tls_stream, peer).await
    });
}
```

Root cause 2: the handshake has no timeout

The TLS future can wait forever for a ClientHello or subsequent handshake bytes. Rate limiting completed requests does not release these pre-request allocations.

Root cause 3: shutdown is observed too late

The service's stop handle is polled after handshake completion, so a pending handshake does not promptly converge during graceful shutdown.

Minimal Reproduction Logic (Isolated Test Environment Only)

On an isolated TLS-enabled node, open a bounded set of TCP connections without sending TLS bytes. Verify whether connections remain pending beyond the intended deadline, then confirm that a fixed build expires them and preserves a legitimate TLS request.

```
for each test connection:
    socket.connect(tls_rpc_address)
    send_nothing()
    hold_for(handshake_timeout + safety_margin)
```

Security Impact

- Legitimate TLS RPC clients cannot connect once descriptors or task capacity are exhausted.
- The RPC task can exit on repeated accept errors, potentially triggering full node shutdown.
- The attacker needs network reachability but no cookie and no valid client certificate unless a front layer requires one before connection admission.

Long-Term Remediation

- Wrap every TLS handshake in a short `tokio::time::timeout`.
- Acquire a global connection permit before spawning and add per-source limits.
- Use `select!` so shutdown cancels pending handshakes immediately.
- Treat `EMFILE` and temporary accept errors with bounded backoff rather than terminating the whole RPC task.

Recommended Regression Tests:

- Hold silent TCP connections beyond the handshake deadline and confirm automatic closure.

- Open more connections than the global limit and confirm deterministic rejection.
- Verify that a valid TLS RPC request succeeds while the limit is under pressure.
- Trigger graceful shutdown with pending handshakes and verify prompt task cleanup.